Neuro-Symbolic LLM-Based Generation and Verification of ATL Transformations

Huy-Hoang Dao, Duc-Quyen Nguyen, and Duc-Hanh Dang*

Faculty of Information Technology, VNU University of Engineering and Technology
VNU Town, Hoa Lac, Hanoi, Vietnam
{23020666|25025048|hanhdd}@vnu.edu.vn

Abstract. Model transformations are widely used in Model-Driven Development (MDD) to automatically derive target models from source models based on transformation rules. Faults in a model transformation may introduce unintended defects in the generated target models. Large Language Models (LLMs) can generate transformations from metamodel descriptions and natural-language transformation requirements, but the resulting transformations may be syntactically valid and execute successfully while still violating metamodel constraints or intended requirements. This paper presents a neuro-symbolic approach for generating and verifying ATL transformations through three successive layers: structural checking, static analysis, and formal verification. Each generated candidate is progressively verified at these three levels, and detected violations are used as feedback for subsequent revision. Evaluated on 15 ATL transformation tasks from LLM4MTLs, the proposed method produces parsable transformations for 100% of the tasks, of which approximately 93% pass the associated functional tests, compared with roughly 53–60% for the considered prompting configurations. These results indicate that integrating verification into generation can improve the functional correctness of LLM-generated ATL transformations on the evaluated tasks.

Keywords: Model transformation · ATL · Large Language Models · Verification · Static analysis · Model finding

1 Introduction

Model transformations play an important role in Model-Driven Development (MDD), where models are used as primary artifacts throughout software development [1,11]. A transformation specifies how models conforming to a source metamodel are mapped to models conforming to a target metamodel, thereby automating the transition between different modeling domains or levels of abstraction. The Atlas Transformation Language (ATL) [9] is a widely used language for specifying such model-to-model transformations. Developing an ATL transformation, however, requires knowledge of the involved metamodels, ATL constructs, OCL [13] expressions used to specify navigation and filtering conditions,

* Corresponding author

and the intended mapping semantics. Recent work has increasingly explored the automation of model transformation development, ranging from the derivation of transformations from explicitly specified correspondences [10] to the use of Large Language Models (LLMs) for generating transformations from metamodels and natural-language requirements [7,8]. These studies demonstrate the feasibility of automated transformation generation, but generated transformations may still contain invalid references to metamodel elements, type inconsistencies, or mappings that do not satisfy the intended transformation requirements.

Several techniques have been developed for analyzing such problems in ATL transformations. Static analysis can identify inconsistencies related to metamodel references, name resolution, scopes, and types [16,17], while formal verification approaches can reason about transformation behavior using constraint-based and symbolic techniques [3,2,12]. However, generation and verification are still largely addressed as separate activities, leaving limited support for an automated process that revises generated ATL candidates based on detected violations and subsequently re-verifies them.

To address this gap, we propose a neuro-symbolic approach combining LLM-based ATL generation with three successive verification layers: structural checking, static analysis, and formal verification. Given source and target metamodels and a natural-language transformation requirement, the method generates structured ATL candidates and uses detected violations as feedback for revision and re-verification. Evaluated on 15 ATL transformation tasks from LLM4MTLs [8], the method produces parsable transformations for all tasks, with 14 passing the associated functional tests, compared with 8–9 for the considered prompting configurations.

This paper makes two main contributions: (i) a neuro-symbolic approach for generating and verifying ATL transformations, structured into three layers: structural checking, static analysis, and formal verification; and (ii) an empirical evaluation of the method on ATL transformation tasks.

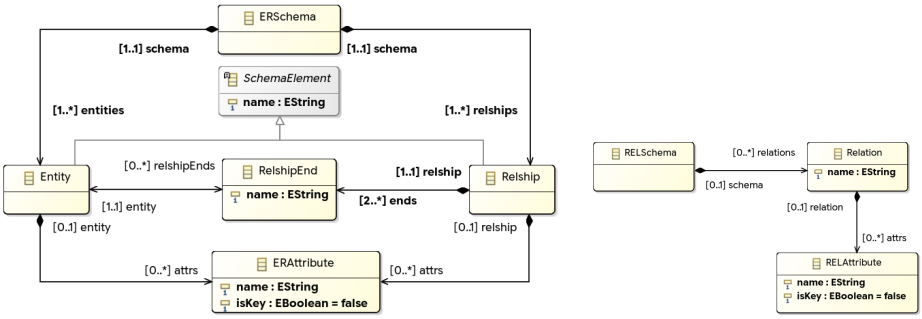
The remainder of this paper is organized as follows. Section 2 introduces the background and motivating example. Section 3 presents the proposed method. Sections 4 and 5 describe ATL candidate generation and verification, respectively. Section 6 presents the experimental evaluation and discusses the results. Section 7 addresses threats to validity. Section 8 reviews related work. The paper ends with a conclusion and an outline of future work.

2 Background and Motivation

This section presents background for this work, focusing on the ER-to-REL transformation as a motivating example.

2.1 Motivation

A metamodel [5] defines the concepts of a modeling domain and the relationships among them, providing the abstract syntax to which models conform. Figure 1 shows the source (ER) and target (REL) metamodels considered in this paper.



(a) ER metamodel (b) REL metamodel

Fig. 1: ER and REL metamodels adapted from [3].

An ER schema contains *entities* and *relationships* with owned *attributes*; *relationship ends* connect relationships to their participating entities. A REL schema contains *relations*, each owning a set of *attributes*. Metamodels also support the definition of constraints over their elements and attributes, expressed as OCL invariants [13]. The following is an example invariant defined in the REL metamodel, which requires attribute names to be unique within each relation:
`context Relation inv uniqueAttr: self.attrs->isUnique(a | a.name)`

2.2 Model Transformation with ATL

ATL [9] is a rule-based language for specifying transformations that map source model elements to target model elements. An ATL module declares the participating source and target models and contains transformation rules. Each matched rule selects source elements in its *from* part and creates target elements in its *to* part. Optional filters restrict rule applications, while bindings specify how values and references of the generated target elements are obtained from the source model.

Listing 1: ATL transformation for the ER-to-REL example

```
module ER2REL; create OUT : REL from IN : ER;
rule S2S {
 from s : ER!ERSchema
 to t : REL!RELSchema (relations <- s.entities->union(s.relships))}
rule E2R {
 from s : ER!Entity
 to t : REL!Relation (name <- s.name, schema <- s.schema)}
rule R2R {
 from s : ER!Relship
 to t : REL!Relation (name <- s.name, schema <- s.schema)}
rule EA2A {
 from att : ER!ERAttribute, ent : ER!Entity (att.entity=ent)
 to t : REL!RELAttribute (name <- att.name, isKey <- att.isKey, relation <- ent)}
rule RA2A {
 from att : ER!ERAttribute, rs : ER!Relship (att.relship=rs)
 to t : REL!RELAttribute (name <- att.name, isKey <- att.isKey, relation <- rs)}
rule RA2AK {
 from att : ER!ERAttribute, rse : ER!RelshipEnd (att.entity=rse.entity and att.isKey=true)
 to t : REL!RELAttribute (name <- att.name, isKey <- att.isKey, relation <- rse.relship)}
```

Listing 1 shows the ATL transformation defined for the ER-to-REL example. Rule `S2S` maps each ER schema to a relational schema. Its `relations` binding collects source entities and relationships, which ATL implicitly resolves to the target relations produced by `E2R` and `R2R`. These two rules preserve the names of the source elements. Rules `EA2A` and `RA2A` transform attributes owned by entities and relationships, respectively, preserving their names and key flags and attaching them to the corresponding relations. Finally, `RA2AK` propagates key attributes from participating entities to the relations representing their relationships, using the corresponding relationship ends.

3 Overview of the Proposed Method

We propose a neuro-symbolic approach for improving the reliability of LLM-generated ATL transformations by integrating LLM-based generation and revision with a multi-layer symbolic verification process, as illustrated in Figure 2.

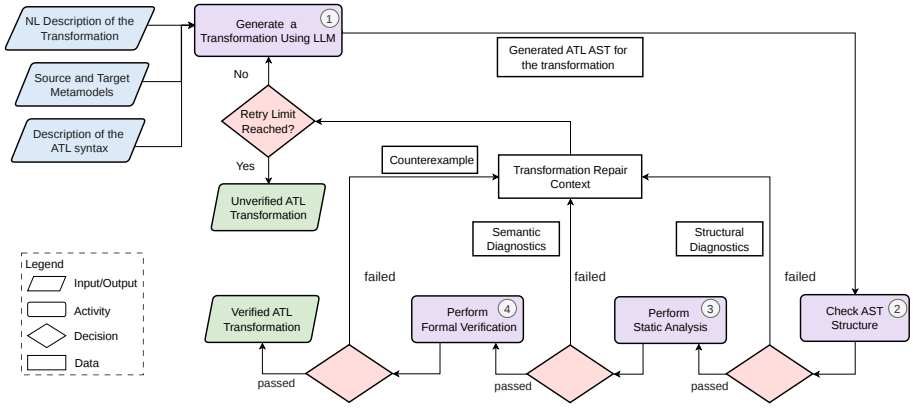


Fig. 2: Overview of the proposed neuro-symbolic approach.

The inputs are a natural-language description of the transformation, the source and target metamodels, and a description of the ATL syntax accompanied by an ATL JSON schema. Each generated candidate undergoes three verification layers: (i) structural checking against the ATL JSON schema; (ii) static analysis of metamodel references and expression types; and (iii) formal verification to determine whether valid source models are transformed into target models satisfying the target metamodel constraints. Only candidates passing all three layers are accepted; otherwise, diagnostics or counterexamples are returned to the LLM as feedback for revision and subsequent verification.

4 ATL Candidate Generation

During ATL candidate generation, the LLM uses metamodel information to construct source-to-target mappings, while the ATL syntax specification describes both the abstract and concrete syntax, guiding how transformation components

are organized and represented. The system prompt for structured ATL generation and the task-specific user prompt for the ER-to-REL motivating example are provided in Appendices A and B, respectively.

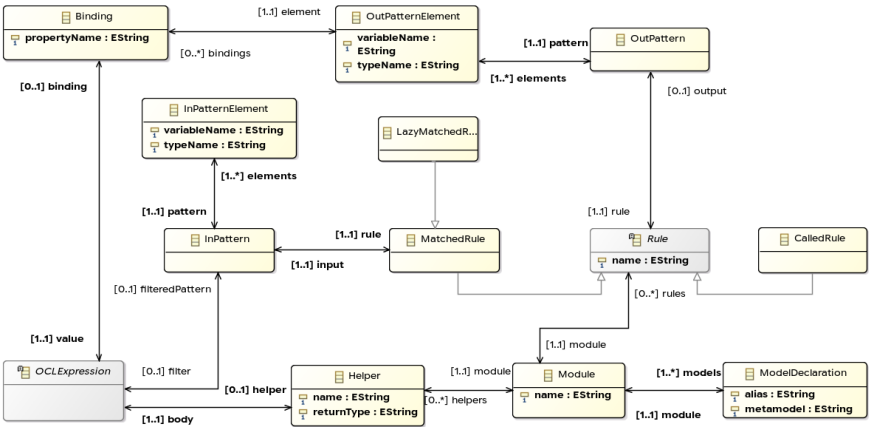


Fig. 3: The simplified metamodel of the ATL abstract syntax.

Figure 3 illustrates the simplified ATL abstract syntax. An ATL transformation is represented by a `Module` containing `ModelDeclaration` elements, transformation rules, and helpers. A `MatchedRule` is automatically applied to matching source elements, whereas `LazyMatchedRule` and `CalledRule` are explicitly invoked. A `MatchedRule` consists of an `InPattern` with typed source elements and an optional OCL filter, and an `OutPattern` whose elements contain `Binding`s that assign target properties using `OCLExpression`s. These expressions are also used in filters and `Helper` bodies. Based on this abstract syntax, we construct an ATL JSON schema to guide structured candidate generation. The same schema is used to check structural validity in the first verification layer.

5 Verification of Generated ATL Transformations

This section describes the three verification layers used to assess generated ATL transformations: structural checking, static analysis, and formal verification.

Structural Checking. The first layer checks whether the JSON representation of each ATL candidate conforms to the ATL JSON schema derived from the ATL abstract syntax.

Static Analysis. The second layer checks whether the ATL transformation conforms to the source and target metamodels [16]. A transformation satisfies this condition when the model types and features referenced by its rules are defined in the corresponding metamodels and the expressions used in the transformation are type consistent.

Formal Verification. The third layer verifies whether valid source models are transformed into target models satisfying the specified target metamodel constraints. We adopt the *transformation model* approach [3] and implement a

higher-order ATL transformation (HOT) to automatically generate transformation models. Figure 4 summarizes this process.

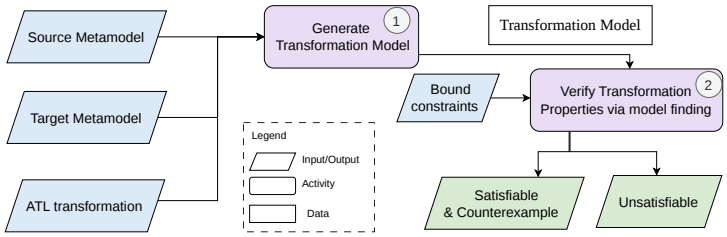


Fig. 4: Overview of the formal verification layer.

The verification first constructs a transformation model from the source and target metamodels, and the ATL transformation. The transformation model combines the concepts of the two modeling domains with trace elements representing the applications of ATL rules. Each ATL rule is represented by a trace class that relates the source elements matched by the rule to the target elements created by that rule. The semantics of the transformation are then sharpened using OCL invariants over this integrated model.

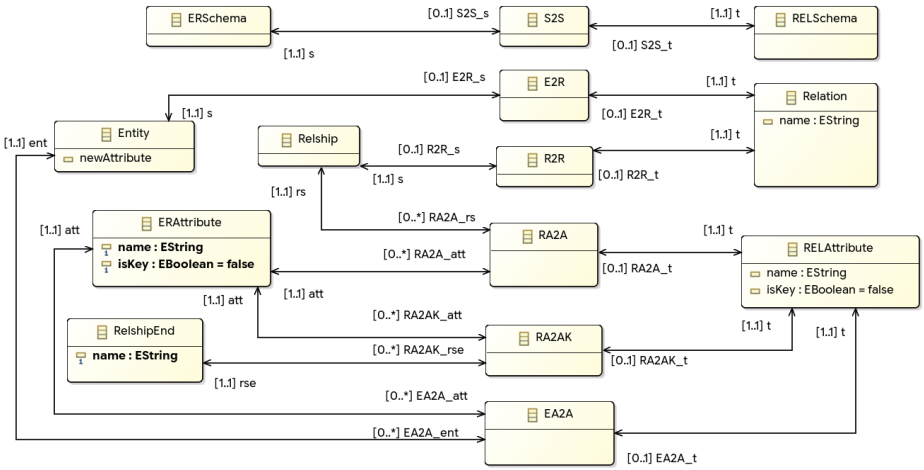


Fig. 5: The ER-to-REL transformation model.

Figure 5 illustrates the ER-to-REL transformation model, which retains the concepts and constraints of both metamodels and introduces trace classes for the six ATL rules in Listing 1. For example, `E2R` relates a source *Entity* to the generated *Relation*, while `RA2AK` relates a source key attribute and a relationship end to the relational attribute generated for the corresponding relationship.

We use eFinder [14] for bounded model finding. The verification problem combines three sets of constraints: source metamodel constraints defining valid source models, OCL constraints representing the ATL transformation semantics, and the negation of the target constraint being verified. Constraints defined in the source and target metamodels are automatically reused, while additional

verification properties can be manually specified through external OCL configurations. The model finder searches for counterexamples in which valid source models yield target models violating the specified constraint. A SAT result identifies a counterexample, whereas UNSAT indicates that none exists within the configured verification bounds.

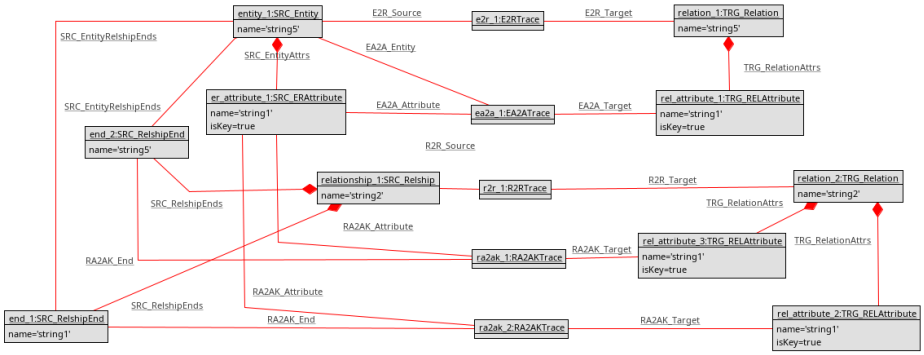


Fig. 6: A counterexample generated by model finding.

Figure 6 shows a counterexample for the ER-to-REL transformation in which a relation created from an ER relationship contains two generated key attributes with the same name, violating the uniqueness constraint of relation attributes. The counterexample identifies the relevant source elements and the corresponding `RA2AK` rule applications. This information is returned to the LLM as verification feedback so that the foreign-key generation can be revised before the candidate is verified again.

Note that our method depends on the capabilities of current model finding tools: the formal verification layer supports only a declarative subset of ATL, including matched rules and non-recursive OCL helpers, as well as `Set` and `Bag` collections, but not `Sequence` or `OrderedSet`. Unsupported constructs, including imperative extensions, called and lazy rules, recursive helpers, and `resolveTemp`, remain outside formal verification, although some can still be represented in the generated candidate and checked by earlier layers.

6 Experimental Evaluation

The proposed method was implemented as a Python-based prototype integrating LLM-based generation, multi-layer verification, and iterative revision.[1] We evaluate it on 15 ATL transformation tasks from LLM4MTLs [8], using 45 functional test cases. The transformations average 96.3 lines of code, 5.5 rules, and 2.2 helpers. The benchmark covers diverse transformation tasks, including direct mappings (`NetworkToGraph`), bidirectional references (`Grafcet2PetriNet`), and more complex mappings involving opposite references, multiplicities, and nested control-flow structures (`DSL2KM3` and `CPL2SPL`).

[1] `https://github.com/vnu-sme/genATLTrafo.git`

Experiments use Gemini 3.6 Flash in its default configuration, with up to five revision iterations. Default verification bounds are set to a maximum of 5 objects per metamodel class and 10 links per reference, denoted $(5, 10)$. Following LLM4MTLs [8], we report: (i) *Parsed*, the number of transformations accepted by the ATL parser; (ii) PPL_s, the average number of parser-reported problems per line of code; and (iii) *Tests Passed*, the number of transformations passing all corresponding functional test cases. We also report average token consumption (*Avg. Tokens*) and end-to-end processing time (*Avg. Time*) per task.

Table 1: Ablation study of verification layers

Configuration	Parsed ↑	PPL_s ↓	Tests Passed ↑	Avg. Time (s) ↓
Without L1	15/15	0.0000	14/15	209.89
Without L2	14/15	0.0004	11/15	128.86
Without L3	15/15	0.0000	12/15	102.64
With all layers	15/15	0.0000	14/15	234.98

Table 1 shows that removing structural checking (L1) does not affect parsing or functional test outcomes, possibly due to the ATL JSON schema used during generation. Removing static analysis (L2) reduces both outcomes, while disabling formal verification (L3) reduces functional test success without affecting parsing. The two additional failures without L3 occur in `DSL2KM3` and `Grafcet2PetriNet`, for which the configuration with all layers found counterexamples and revised the generated transformations. These findings suggest that static analysis and formal verification can improve functional correctness on the evaluated tasks.

Table 2: Comparison with Prompting Strategies

Method	Parsed ↑	PPL_s ↓	Tests Passed ↑	Avg. Tokens (k) ↓	Avg. Time (s) ↓
Baseline	0.87 (13/15)	0.0035	0.53 (8/15)	7.75	46.29
Few-shot (FS)	0.87 (13/15)	0.0007	0.53 (8/15)	8.13	39.76
Grammar (GR)	0.93 (14/15)	0.0002	0.60 (9/15)	9.72	33.59
FS + GR	0.93 (14/15)	0.0002	0.60 (9/15)	9.92	34.48
Proposed Method	**1.00 (15/15)**	**0.0000**	**0.93 (14/15)**	**64.74**	**234.98**

To evaluate the proposed method, we compare it with four prompting strategies adopted from LLM4MTLs [8]: (i) Baseline, using only transformation requirements and metamodels; (ii) Few-shot (FS), adding three ATL transformation examples; (iii) Grammar (GR), incorporating the ATL grammar; and (iv) FS+GR, combining both strategies. As shown in Table 2, the proposed method achieves higher parsing and functional test success, with 93% (14/15) of transformations passing all corresponding tests, compared with 53–60% (8–9/15) for the prompting strategies. Despite their near-zero parser-reported problem rates, GR and FS+GR still achieve limited functional test success, indicating that syntactic quality alone does not guarantee functional correctness. These improvements come at the cost of substantially higher token consumption and processing time due to verification and iterative revision.

Table 3 reports the per-task token consumption, processing time, revision counts, verification outcomes, and functional test results. At the default bounds, eight final candidates passed all enabled, supported verification checks, while seven remained unverified due to unsupported constructs. For `Grafcet2PetriNet`

Table 3: Per-task experimental results of the proposed method

Transformation Task	Tokens (k)	Time (s)	Revisions	Verification Outcome	Functional Tests
`Class2Interface`	18.140	96.859	0	UNSAT	PASS
`Document2Report`	19.152	75.829	0	UNSAT	PASS
`Item2Product`	17.585	50.472	0	UNSAT	PASS
`User2Account`	17.030	30.951	0	UNSUPPORTED	PASS
`NetworkToGraph`	19.097	27.690	0	UNSUPPORTED	PASS
`FamiliesToPersons`	21.277	72.017	0	UNSAT	PASS
`AmaltheaToAscet`	18.123	34.539	0	UNSAT	PASS
`BibTeX2DocBook`	119.902	346.967	2	UNSUPPORTED	PASS
`XML2DSL`	200.523	606.175	4	UNSUPPORTED	FAIL
`PetriNet2Grafcet`	22.395	143.998	0	UNSAT	PASS
`Grafcet2PetriNet`	49.624	332.097	1	SAT $\rightarrow$ UNSAT	PASS
`DSL2KM3`	139.392	518.316	3	SAT $\rightarrow$ UNSAT	PASS
`IEEE1471_2_MoDAF`	75.305	607.174	1	UNSUPPORTED	PASS
`Make2Ant`	30.477	100.565	0	UNSUPPORTED	PASS
`CPL2SPL`	202.775	481.105	3	UNSUPPORTED	PASS

and `DSL2KM3`, verification outcomes changed from SAT to UNSAT following revision.

To assess sensitivity to verification bounds, we rechecked these eight candidates at bounds $(7, 14)$ and $(9, 18)$, finding no counterexamples for the enabled properties. For the unrepaired `DSL2KM3` candidate, increasing the bounds from $(1, 2)$ to $(2, 4)$ exposed a counterexample. At bounds $(15, 30)$, four tasks reached the 900-second per-task timeout.

Discussion. The evaluation shows that verification-guided revision can improve generated transformations beyond syntactic validity alone. However, the absence of counterexamples applies only to the considered target metamodel constraints within the configured verification bounds. Formal verification is limited to the supported ATL/OCL subset, while verification feedback does not guarantee successful LLM revision, potentially requiring multiple attempts.

7 Threats to Validity

Several factors may influence the experimental results. First, LLM nondeterminism may lead to different candidates and verification outcomes across runs, while result stability has not been systematically assessed. Second, the evaluation covers only 15 ATL transformation tasks and one LLM configuration, limiting generalizability. Third, parser-oriented measures mainly capture syntactic properties, while functional-test results depend on test coverage. Finally, the comparison with the prompting strategies evaluates the overall generation, verification, and revision process rather than the individual verification layers.

8 Related Work

Research related to this work can be grouped into three directions: automated generation, error detection and repair, and formal verification of model trans-

formations. Early approaches explored ATL generation through metamodel correspondences [10], reusable transformation templates [15], and transformation-by-example techniques [6]. More recently, Iqbal and Al-Azzoni [7] investigated LLM-based ATL generation, while Jiang et al. [8] introduced LLM4MTLs, which generates and evaluates model transformations under different prompting strategies. Grammar prompting [21] has also been explored to improve domain-specific language generation. Although these approaches demonstrate the feasibility of automated transformation generation, their evaluation does not establish the absence of constraint violations.

A second line of work focuses on detecting and repairing ATL transformation errors. Static analysis techniques [16] identify problems involving metamodel references, name resolution, scopes, and types. Subsequent approaches provide automated repair through quick fixes and speculative analysis [17], search-based techniques [20], and test-based, multi-objective patch generation [19]. However, eliminating detected errors does not necessarily guarantee that the generated target models satisfy their metamodel constraints.

Formal verification techniques address transformation behavior beyond static correctness. Büttner et al. [3] proposed the *transformation model* approach, representing ATL rule semantics and metamodel constraints in a common model to support counterexample generation. Other approaches employ SMT solving [2], symbolic execution and contract reasoning [12], or translation validation [4]. Transformation-model-based verification has also been applied in an industrial setting [18], while eFinder [14] supports model finding for Ecore models constrained by OCL. However, such verification results are generally used to identify or diagnose violations rather than being incorporated into an automated revision-and-re-verification process.

Our work addresses these limitations by combining automated ATL generation and revision with systematic verification of structural validity, metamodel consistency, and target-model constraints. This places our work at the intersection of the three research directions discussed above.

9 Conclusion and Future Work

This paper presented a neuro-symbolic approach that combines LLM-based generation of ATL transformations with structural checking, static analysis, and formal verification, with detected violations guiding revision. Evaluated on 15 ATL transformation tasks from LLM4MTLs, the method produced parsable transformations for all tasks, with approximately 93% passing the associated functional tests, compared with 53–60% for the considered prompting configurations. These results indicate that integrating verification into the generation process can improve the functional correctness of LLM-generated ATL transformations on the evaluated tasks.

Future work will extend the formal verification layer to support a broader range of ATL/OCL constructs and explore its application to other model transformation languages. We also plan to improve candidate revision through more

effective use of diagnostics and counterexamples. To further evaluate the approach, we will consider additional LLMs and systematically assess the stability of experimental results. Finally, we plan to evaluate the approach on larger and more diverse benchmarks, including metamodel variants created through semantics-preserving non-structural modifications.

References

1. Brambilla, M., Cabot, J., Wimmer, M.: Model-Driven Software Engineering in Practice. Synthesis Lectures on Software Engineering, Morgan & Claypool, 2 edn. (2017)
2. Büttner, F., Egea, M., Cabot, J.: On Verifying ATL Transformations Using "Off-the-Shelf" SMT Solvers. In: Proc. 15th Int. Conf. Model Driven Engineering Languages and Systems (MODELS). pp. 432–448. LNCS 7590, Springer (2012)
3. Büttner, F., Egea, M., Cabot, J., Gogolla, M.: Verification of ATL Transformations Using Transformation Models and Model Finders. In: Proc. 14th Int. Conf. Formal Methods and Software Engineering (ICFEM). pp. 198–213. LNCS 7635, Springer (2012)
4. Cheng, Z., Monahan, R., Power, J.F.: A Sound Execution Semantics for ATL via Translation Validation. In: Proc. 8th Int. Conf. Model Transformation (ICMT). pp. 133–148. LNCS 9152, Springer (2015)
5. García-Magariño, I., Fuentes-Fernández, R., Gómez-Sanz, J.J.: Guideline for the Definition of EMF Metamodels Using an Entity-Relationship Approach. Inf. Softw. Technol. **51**(8), 1217–1230 (2009)
6. García-Magariño, I., Gómez-Sanz, J.J., Fuentes-Fernández, R.: Model Transformation By-Example: An Algorithm for Generating Many-to-Many Transformation Rules in Several Model Transformation Languages. In: Proc. 2nd Int. Conf. Model Transformation (ICMT). pp. 52–66. LNCS 5563, Springer (2009)
7. Iqbal, S., Al-Azzoni, I.: Using LLMs for Writing ATL Model Transformation Code. In: Proc. 26th Int. Conf. Information Technology (ACIT). pp. 304–308. IEEE (2025)
8. Jiang, B., Hagel, N., Cheng, H., Jutz, B., Lange, A., Zhang, W., Sharma, R., Reussner, R., Koziolek, A.: LLM4MTLs: Automated Generation and Empirical Evaluation of Model Transformation Languages. J. Object Technol. **25**(3), 3:281–294 (2026)
9. Jouault, F., Allilaire, F., Bézivin, J., Kurtev, I.: ATL: A Model Transformation Tool. Sci. Comput. Program. **72**(1–2), 31–39 (2008)
10. Lano, K., Fang, S.: Automated Synthesis of ATL Transformations from Metamodel Correspondences. In: Proc. 8th Int. Conf. Model-Driven Engineering and Software Development (MODELSWARD). pp. 263–270. SCITEPRESS (2020)
11. Mens, T., Van Gorp, P.: A Taxonomy of Model Transformation. Electron. Notes Theor. Comput. Sci. **152**, 125–142 (2006)
12. Oakes, B.J., Troya, J., Lúcio, L., Wimmer, M.: Full Contract Verification for ATL Using Symbolic Execution. Softw. Syst. Model. **17**(3), 815–849 (2018)
13. Object Management Group: Object Constraint Language (OCL), Version 2.4. OMG Specification formal/2014-02-03, Object Management Group (2014)
14. Sánchez Cuadrado, J., Gogolla, M.: Model Finding in the EMF Ecosystem. J. Object Technol. **19**(2), 10:1–21 (2020)

15. Sánchez Cuadrado, J., Guerra, E., de Lara, J.: Flexible Model-to-Model Transformation Templates: An Application to ATL. J. Object Technol. **11**(2), 4:1–28 (2012)
16. Sánchez Cuadrado, J., Guerra, E., de Lara, J.: Static Analysis of Model Transformations. IEEE Trans. Softw. Eng. **43**(9), 868–897 (2017)
17. Sánchez Cuadrado, J., Guerra, E., de Lara, J.: Quick Fixing ATL Transformations with Speculative Analysis. Softw. Syst. Model. **17**(3), 779–813 (2018)
18. Selim, G.M.K., Büttner, F., Cordy, J.R., Dingel, J., Wang, S.: Automated Verification of Model Transformations in the Automotive Industry. In: Proc. 16th Int. Conf. Model Driven Engineering Languages and Systems (MODELS). pp. 690–706. LNCS 8107, Springer (2013)
19. VaraminyBahnemiry, Z., Galasso, J., Belharbi, K., Sahraoui, H.A.: Automated patch generation for fixing semantic errors in atl transformation rules. In: Proc. 24th Int. Conf. Model Driven Engineering Languages and Systems (MODELS). pp. 13–23. IEEE (2021)
20. VaraminyBahnemiry, Z., Galasso, J., Sahraoui, H.A.: Fixing multiple type errors in model transformations with alternative oracles to test cases. J. Object Technol. **20**(3), 9:1–14 (2021)
21. Wang, B., Wang, Z., Wang, X., Cao, Y., Saurous, R.A., Kim, Y.: Grammar Prompting for Domain-Specific Language Generation with Large Language Models. In: Adv. Neural Inf. Process. Syst. vol. 36, pp. 65030–65055. Curran Associates (2023)

A System Prompt for ATL Candidate Generation

The following system prompt provides the LLM with the constraints used to generate structured ATL candidates. The placeholder <INSERT_SCHEMA_HERE> is replaced at runtime by the ATL JSON schema described in Section 4.

Listing 2: System prompt for ATL candidate generation

```
You are an expert in Model-Driven Engineering, ATL, OCL,
and AST construction.

TASK

Translate the user's transformation specification into a JSON AST
that strictly conforms to the supplied ATL JSON schema.

The ATL AST schema is:
<INSERT_SCHEMA_HERE>

OUTPUT

- Output exactly one valid JSON object.
- Do not output ATL source, Markdown, or explanations.
- Follow the schema exactly: include all required fields and
  invent no fields.

MODULE AND TYPES

- Build a complete ATL module with all required
  ModelDeclaration objects.
- Types MUST use the exact EPackage name:
  <EPackage>!<EClass>.
- For primitive EDataTypes, always use bare OCL types
  such as Boolean, Integer, Real, and String.

RULE SELECTION

- Use MatchedRule for normal declarative mappings.
- Use LazyMatchedRule or CalledRule only when explicitly required.

BINDINGS AND TRACE RESOLUTION

- Every explicitly required target assignment must appear
  as a Binding.
- Binding property names and values must be type-compatible
  with the corresponding target Ecore features.
- Use ATL implicit trace resolution for reference-valued
  bindings when compatible source elements are mapped.

OCL AND SUPPORTED SUBSET
```

```
- Represent every expression using schema-defined OCL AST
  nodes. Never emit raw OCL text.
- Supported primitive types are Integer, Real, Boolean,
  String, and Null.
- Use only the supported logical and arithmetic operators
  defined by the schema.
- Supported collections are Set and Bag.
- Supported iterators are forAll, exists, select, reject,
  collect, and isUnique.
- Use oclIsUndefined() for null checks. Null literal
  comparisons are not supported.

ECORE COMPLIANCE AND MULTIPLICITY

- Verify every EClass, feature, type, and multiplicity
  against the supplied Ecore before emitting it.
- Never guess, rename, or approximate identifiers.
  Preserve exact Ecore spelling and capitalization.
- Never apply collection operations to single-valued
  features.
```

B User Prompt for the ER-to-REL Transformation

The following user prompt specifies the transformation requirements for the ER-to-REL example introduced in Section 2. Together with the source and target metamodels, this prompt is provided to the LLM as the task-specific input.

Listing 3: User prompt for the ER-to-REL transformation

```
Create an ATL transformation that converts an
Entity-Relationship (ER) model into a Relational database
model based on the following requirements:

1. Schema Mapping:
   Transform the overall ER diagram into a relational schema.
   This schema must act as the container for all generated
   tables.

2. Table Generation:
   Both entities and relationships in the ER model should be
   converted into their own relational tables.

3. Attribute Mapping:
   Simple attributes belonging to entities and relationships
   should translate directly into columns within their
   respective tables.

4. Foreign Key Resolution:
   For each relationship, identify every participating entity
   and copy each of its primary-key attributes into the
   relational table generated for that relationship. The
   copied attributes represent foreign keys and preserve
   their names and key information.
```